

Guide d'Administration — SGH (Système de Gestion des Hypothèques)

Société Ivoirienne de Banque (SIB) Version 1.0 — Août 2026

Table des matières

- 1. [Vue d'ensemble du système](#)
- 2. [Installation locale \(développement\)](#)
- 3. [Déploiement en production](#)
- 4. [Gestion des utilisateurs](#)
- 5. [Maintenance courante](#)
- 6. [Sauvegarde de la base de données](#)
- 7. [Monitoring et logs](#)
- 8. [Résolution des problèmes courants](#)
- 9. [Sécurité](#)
- 10. [Référence des variables d'environnement](#)

1. Vue d'ensemble du système

Le SGH est une application web de gestion du portefeuille hypothécaire de la SIB. Elle permet le suivi des garanties immobilières, la réévaluation des biens selon la Circulaire 04-2017 de la BCEAO, la génération d'alertes automatiques et l'export de rapports réglementaires.

Architecture

Composant	Technologie	Environnement de production
Backend API	Node.js / Express / TypeScript	Railway — sgh-backend
Frontend	Next.js 14	Vercel — sgh-frontend
Base de données	SQLite (Prisma ORM)	Volume Railway (hypothèque.db)
Code source	Monorepo GitHub	https://github.com/Ibrahim044653/Garantie (branche main)

URLs de production

- **Frontend** : <https://sgh-frontend.vercel.app>
- **Backend API** : <https://sgh-backend-production-297b.up.railway.app>
- **Health check** : <https://sgh-backend-production-297b.up.railway.app/api/health>

Structure du dépôt

```
Hypothèque/  
├── backend/           # API Express + Prisma  
│   ├── src/  
│   │   ├── controllers/ # Logique métier  
│   │   ├── routes/      # Définition des routes  
│   │   └── middleware/   # Auth, validation
```

```
| | | └─ services/      # Logger, alertes
| | └─ prisma/
| |   └─ schema.prisma  # Schéma BDD
| |   └─ seed.ts        # Données initiales
| |   └─ migrations/    # Historique des migrations
| └─ uploads/          # Pièces jointes PDF
└─ frontend/          # Application Next.js
└─ docs/              # Documentation
└─ railway.toml       # Configuration Railway
```

2. Installation locale (développement)

Prérequis

- **Node.js** >= 20.0.0 (vérifier avec `node --version`)
- **npm** >= 9 (inclus avec Node.js)
- **Git**
- Un éditeur de code (VS Code recommandé)

Étapes d'installation

1. Cloner le dépôt

```
git clone https://github.com/Ibrahim044653/Garantie.git
cd Garantie
```

2. Installer les dépendances

```
# Dépendances backend
cd backend
npm install

# Dépendances frontend (dans un autre terminal)
cd frontend
npm install
```

3. Configurer les variables d'environnement

Backend — créer le fichier `backend/.env` :

```
NODE_ENV=development
PORT=3001
DATABASE_URL="file:./prisma/hypothèque.db"
JWT_SECRET=votre_secret_local_dev
JWT_EXPIRES_IN=1h
FRONTEND_URL=http://localhost:3000
```

Frontend — créer le fichier `frontend/.env.local` :

```
BACKEND_URL=http://localhost:3001
NEXT_PUBLIC_API_URL=http://localhost:3001/api
```

4. Initialiser la base de données

```
cd backend

# Créer la base et appliquer le schéma (première installation)
npx prisma migrate dev --name init

# Charger les données initiales (comptes démo + données exemples)
npx prisma db seed
```

5. Démarrer les serveurs

Dans deux terminaux distincts :

```
# Terminal 1 – Backend (port 3001)
cd backend
npm run dev

# Terminal 2 – Frontend (port 3000)
cd frontend
npm run dev
```

L'application est accessible sur <http://localhost:3000>.

Outils de développement utiles

```
# Prisma Studio – interface visuelle de la BDD
cd backend && npx prisma studio

# Réinitialiser complètement la BDD (détruit les données !)
cd backend && npm run db:reset
```

3. Déploiement en production

Le déploiement est entièrement automatisé via CI/CD. **Tout push sur la branche `main` déclenche un redéploiement automatique** du backend (Railway) et du frontend (Vercel).

3.1 Backend — Railway

Paramètres de déploiement (`railway.toml`)

```
[build]
builder = "nixpacks"

[deploy]
```

```
startCommand = "cd backend && npx prisma migrate deploy && npx prisma db seed && npm start"
healthcheckPath = "/api/health"
healthcheckTimeout = 100
restartPolicyType = "ON_FAILURE"
restartPolicyMaxRetries = 5
```

Séquence de démarrage automatique :

1. `npx prisma migrate deploy` — applique les migrations en attente
2. `npx prisma db seed` — crée/met à jour les comptes par défaut (idempotent via `upsert`)
3. `npm start` — démarre le serveur Express
4. À l'écoute : génération des alertes au démarrage, puis toutes les 24h

Variables d'environnement Railway

À configurer dans le tableau de bord Railway → Service `sgh-backend` → onglet **Variables** :

Variable	Valeur
NODE_ENV	production
PORT	3001
JWT_SECRET	sgh_sib_prod_2026_secret_key
JWT_EXPIRES_IN	1h
FRONTEND_URL	https://sgh-frontend.vercel.app,https://frontend-seven-lemon-99.vercel.app

Important : Ne jamais versionner le `JWT_SECRET` dans le dépôt Git. Le modifier uniquement via le tableau de bord Railway.

Déclencher un redéploiement manuel

```
# Via Railway CLI
railway up

# Ou via l'interface Railway : bouton "Redeploy" sur le dernier déploiement
```

3.2 Frontend — Vercel

Configuration Vercel

- **Framework** : Next.js (détection automatique)
- **Dossier racine** : `frontend/`
- **Branche de production** : `main`

Variables d'environnement Vercel

À configurer dans le tableau de bord Vercel → Projet `sgh-frontend` → **Settings** → **Environment Variables** :

Variable	Valeur
BACKEND_URL	https://sgh-backend-production-297b.up.railway.app

NEXT_PUBLIC_API_URL	https://sgh-backend-production-297b.up.railway.app/api
---------------------	--

`NEXT_PUBLIC_*` est exposé côté client. Ne jamais y mettre de secret.

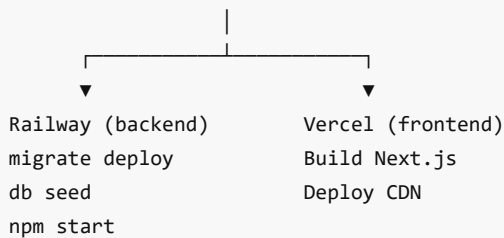
Déclencher un redéploiement Vercel

```
# Via Vercel CLI
vercel --prod

# Ou déclencher automatiquement via un push Git sur main
```

3.3 Flux de déploiement complet

Développeur → git push origin main



4. Gestion des utilisateurs

4.1 Comptes par défaut (seed)

Ces comptes sont créés automatiquement au premier démarrage via `prisma db seed` :

Email	Mot de passe	Rôle
admin@banque.sn	Admin@1234	ADMIN
gestionnaire@banque.sn	Gest@1234	GESTIONNAIRE_GARANTIES
risques@banque.sn	Risques@1234	RESPONSABLE_RISQUES
engagements@banque.sn	Engag@1234	ENGAGEMENTS
audit@banque.sn	Audit@1234	AUDIT_INTERNE

Action impérative en production : Modifier tous les mots de passe par défaut immédiatement après le premier déploiement.

4.2 Rôles et permissions

Rôle	Description	Accès
ADMIN	Administrateur système	Accès total, gestion des utilisateurs
GESTIONNAIRE_GARANTIES	Gestionnaire de garanties	CRUD hypothèques, upload PJ, historique

RESPONSABLE_RISQUES	Responsable des risques	Lecture, réévaluation, alertes, rapports
ENGAGEMENTS	Service engagements	Consultation du portefeuille
AUDIT_INTERNE	Auditeur interne	Lecture seule, accès complets aux rapports

4.3 API d'administration des utilisateurs

Toutes les routes nécessitent un token JWT valide avec le rôle `ADMIN`.

En-tête requis :

```
Authorization: Bearer <token_jwt>
```

Lister tous les utilisateurs

```
GET /api/admin/users
```

Réponse : tableau d'objets utilisateur (sans les mots de passe).

Créer un utilisateur

```
POST /api/admin/users
Content-Type: application/json

{
  "email": "nouveau@banque.sn",
  "password": "MotDePasse@Securise1",
  "nom": "Dupont",
  "prenom": "Jean",
  "role": "GESTIONNAIRE_GARANTIES"
}
```

Valeurs valides pour `role` : `ADMIN`, `GESTIONNAIRE_GARANTIES`, `RESPONSABLE_RISQUES`, `ENGAGEMENTS`, `AUDIT_INTERNE`.

Modifier un utilisateur

```
PUT /api/admin/users/:id
Content-Type: application/json

{
  "email": "nouveau.email@banque.sn",
  "role": "RESPONSABLE_RISQUES"
}
```

Tous les champs sont optionnels. Le mot de passe est rehashé automatiquement si fourni.

Supprimer un utilisateur

```
DELETE /api/admin/users/:id
```

Attention : la suppression est irréversible. Préférer la modification du rôle pour désactiver un compte.

4.4 Authentification MFA (TOTP)

Le SGH supporte le MFA via Google Authenticator ou équivalent TOTP.

Endpoint	Description
GET /api/auth/mfa/setup	Génère le QR code d'enrôlement
POST /api/auth/mfa/confirm	Confirme l'activation avec un code TOTP
POST /api/auth/mfa/validate	Valide le code TOTP lors de la connexion
DELETE /api/auth/mfa/disable	Désactive le MFA pour le compte

5. Maintenance courante

5.1 Appliquer une migration de schéma

Lors d'une modification du fichier `prisma/schema.prisma` :

```
# En développement – crée et applique la migration
cd backend
npx prisma migrate dev --name description_de_la_modification

# En production (Railway CLI) – applique les migrations en attente
railway run npx prisma migrate deploy
```

En production, `prisma migrate deploy` est exécuté automatiquement à chaque démarrage du service. Une intervention manuelle n'est nécessaire qu'en cas d'urgence.

5.2 Re-seeder manuellement

```
# Via Railway CLI (sans redéployer)
railway run npx prisma db seed
```

Le seed est **idempotent** : il utilise `upsert` et ne crée pas de doublons. Il recrée les comptes par défaut s'ils ont été supprimés.

5.3 Réinitialiser la base de données

Opération destructive — à n'effectuer qu'en développement ou après accord explicite.

```
cd backend
npm run db:reset
# Équivalent à : prisma migrate reset --force && prisma db seed
```

5.4 Gestion des fichiers uploadés (PJ expertises)

Les pièces jointes PDF sont stockées dans le dossier `/uploads` à l'intérieur du container Railway.

```
# Lister les fichiers uploadés
railway run ls /app/backend/uploads

# Copier un fichier localement (via Railway CLI v3+)
railway run cat /app/backend/uploads/fichier.pdf > fichier_local.pdf
```

Attention : le dossier `/uploads` est dans le container et **non persisté entre redéploiements** si Railway ne monte pas de volume. Vérifier la configuration du volume dans le tableau de bord Railway.

5.5 Mise à jour des dépendances

```
# Vérifier les mises à jour disponibles
cd backend && npm outdated
cd frontend && npm outdated

# Mettre à jour (tester localement avant de pousser)
npm update
```

6. Sauvegarde de la base de données

La base SQLite est un fichier unique `hypothèque.db` situé dans `backend/prisma/` (développement) ou sur le volume Railway (production).

6.1 Sauvegarde manuelle via Railway CLI

```
# Copier la BDD depuis le container Railway
railway run cat /app/backend/prisma/hypothèque.db > backup_$(date +%Y%m%d_%H%M%S).db
```

6.2 Export SQL complet

```
# En local
cd backend
npx prisma db pull # synchroniser le schéma si nécessaire

# Dump SQLite vers SQL
sqlite3 prisma/hypothèque.db .dump > backup_$(date +%Y%m%d).sql
```

6.3 Restauration

```
# Arrêter le service Railway temporairement, puis :
railway run bash -c "cp /chemin/vers/backup.db /app/backend/prisma/hypothèque.db"
```


6.4 Recommandations de sauvegarde

Fréquence	Type	Rétention
Quotidienne	Export SQL automatisé (cron)	30 jours
Hebdomadaire	Copie du fichier .db	3 mois
Avant chaque déploiement majeur	Snapshot manuel	À conserver

Recommandation : Mettre en place un script cron externe (GitHub Actions planifié ou service tiers) pour automatiser les sauvegardes quotidiennes vers un stockage S3 ou équivalent.

7. Monitoring et logs

7.1 Logs Railway en temps réel

```
# Suivre les logs du service backend
railway logs

# Filtrer sur les erreurs
railway logs | grep -i error

# Filtrer sur les alertes
railway logs | grep -i alert
```

7.2 Interprétation des logs (Winston)

Le logger est configuré dans backend/src/services/logger.ts . Les niveaux de log sont :

Niveau	Signification	Action requise
info	Opération normale	Aucune
warn	Situation anormale non bloquante	Surveiller
error	Erreur — fonctionnalité impactée	Intervention requise

Exemples de logs normaux au démarrage :

```
info: Server running on port 3001
info: Alert generation completed at startup
info: Daily alert generation completed
```

7.3 Health Check

Le endpoint /api/health retourne l'état du service :

```
curl https://sgh-backend-production-297b.up.railway.app/api/health
# Réponse attendue : {"status":"ok","timestamp":"2026-08-18T...", "version":"1.0.0"}
```

7.4 Surveillance des alertes métier

Les alertes sont générées automatiquement :

- **Au démarrage du service**
- **Toutes les 24h** via `setInterval`

Types d'alertes surveillés : péremption d'inscription, dépassement du LTV, échéance de prêt approchante.

8. Résolution des problèmes courants

8.1 Le backend ne démarre pas

Symptôme : Le health check renvoie une erreur 502 ou timeout.

Vérifications :

```
railway logs # Chercher les erreurs au démarrage
```

Causes fréquentes :

- `JWT_SECRET` non défini → vérifier les variables d'environnement Railway
- Échec de migration Prisma → vérifier les logs de migration
- Conflit de port → vérifier que `PORT=3001` est bien défini

8.2 Erreur CORS en production

Symptôme : Le frontend affiche `"Not allowed by CORS"` dans la console navigateur.

Solution :

1. Vérifier que `FRONTEND_URL` dans Railway contient bien l'URL Vercel en cours
2. Si une nouvelle URL de déploiement Vercel est créée, l'ajouter séparée par une virgule :

```
FRONTEND_URL=https://sgh-frontend.vercel.app,https://nouvelle-url.vercel.app
```

8.3 Erreur "Token expired" ou "Invalid token"

Symptôme : Les utilisateurs sont déconnectés prématurément ou reçoivent des erreurs 401.

Vérifications :

- `JWT_SECRET` identique entre tous les redéploiements (ne pas le régénérer sans déconnecter tous les utilisateurs)
- `JWT_EXPIRES_IN` configuré (défaut : `1h`)

Solution : Les utilisateurs doivent se reconnecter. Si le problème persiste, vérifier que `JWT_SECRET` n'a pas changé.

8.4 Seed qui échoue

Symptôme : Erreur au démarrage sur `prisma db seed`.

Solution :

```
# Vérifier manuellement
railway run npx prisma db seed

# Si la BDD est corrompue, réinitialiser (⚠ perte de données)
railway run npx prisma migrate reset --force
```

8.5 Les alertes ne sont pas générées

Symptôme : Aucune alerte visible dans l'interface alors que des hypothèques arrivent à échéance.

Vérifications :

```
railway logs | grep -i "alert generation"
```

Solution : Si le log "Alert generation completed at startup" est absent, redémarrer le service Railway.

8.6 Fichiers uploadés inaccessibles

Symptôme : Les PDF d'expertise renvoient une erreur 404.

Cause : Le dossier /uploads n'est pas monté sur un volume persistant.

Solution : Dans le tableau de bord Railway, vérifier la configuration du volume et que le chemin /app/backend/uploads est bien monté. Après correction, les fichiers devront être ré-uploadés.

8.7 Le frontend Vercel affiche une page blanche

Symptôme : L'application charge mais l'interface est vide.

Vérifications :

1. Vérifier les variables d'environnement Vercel (NEXT_PUBLIC_API_URL)
2. Consulter les logs de build Vercel
3. Vérifier que le backend répond bien via le health check

9. Sécurité

9.1 Mesures en place

Mesure	Implémentation
Authentification JWT	Token signé, expiration 1h
Hashage des mots de passe	bcryptjs, 10 rounds
MFA	TOTP via speakeasy (Google Authenticator)
Protection des en-têtes HTTP	Helmet.js activé
CORS restrictif	Limité aux URLs FRONTEND_URL
Validation des entrées	express-validator sur tous les endpoints
Taille des uploads	Limitée à 10 Mo

9.2 Bonnes pratiques opérationnelles

- 1. **Mots de passe par défaut** : Les modifier immédiatement après le premier déploiement.
- 2. **JWT_SECRET** : Utiliser une chaîne aléatoire d'au moins 32 caractères. Ne jamais la partager ni la versionner.
- 3. **MFA** : Activer le MFA pour tous les comptes ADMIN et RESPONSABLE_RISQUES.
- 4. **Rotation du JWT_SECRET** : En cas de compromission suspectée, modifier le secret dans Railway → toutes les sessions actives seront invalidées.
- 5. **Accès Railway** : Restreindre l'accès au projet Railway aux membres de l'équipe technique uniquement.
- 6. **Revue des accès** : Auditer trimestriellement la liste des utilisateurs actifs via `GET /api/admin/users` .

9.3 Procédure en cas de compromission

- 1. **Modifier immédiatement** le `JWT_SECRET` dans Railway → redéploiement automatique → toutes les sessions sont invalidées
- 2. **Identifier** le compte compromis via les logs Railway
- 3. **Supprimer ou désactiver** le compte via `DELETE /api/admin/users/:id`
- 4. **Vérifier** l'historique des modifications dans la table `HistoriqueValeur`
- 5. **Notifier** la DSI de la SIB

10. Référence des variables d'environnement

Backend (Railway)

Variable	Obligatoire	Défaut	Description
NODE_ENV	Oui	development	Environnement d'exécution
PORT	Oui	3001	Port d'écoute du serveur
DATABASE_URL	Non	file:./prisma/hypothèque.db	Chemin SQLite (géré par Prisma)
JWT_SECRET	Oui	—	Clé de signature JWT (obligatoire en production)
JWT_EXPIRES_IN	Non	1h	Durée de vie des tokens JWT
FRONTEND_URL	Oui	http://localhost:3000	URLs autorisées par CORS (séparées par virgule)

Frontend (Vercel)

Variable	Obligatoire	Description
BACKEND_URL	Oui	URL du backend pour les appels serveur Next.js
NEXT_PUBLIC_API_URL	Oui	URL de l'API exposée côté client

Annexe — Commandes de référence rapide

```
# --- Railway CLI ---
railway logs                # Logs en temps réel
railway run npx prisma db seed  # Re-seeder la BDD
```

```
railway run npx prisma migrate deploy      # Appliquer les migrations
railway run npx prisma studio              # Interface visuelle BDD
railway up                                 # Redéployer manuellement

# --- Développement local ---
cd backend && npm run dev                   # Démarrer le backend (hot reload)
cd frontend && npm run dev                  # Démarrer le frontend
cd backend && npx prisma migrate dev        # Créer et appliquer une migration
cd backend && npx prisma db seed            # Charger les données initiales
cd backend && npm run db:reset              # Réinitialiser la BDD (⚠ destructif)
cd backend && npx prisma studio             # Interface visuelle locale

# --- Health check ---
curl https://sgh-backend-production-297b.up.railway.app/api/health

# --- API utilisateurs (remplacer TOKEN par un JWT admin valide) ---
curl -H "Authorization: Bearer TOKEN" https://sgh-backend-production-297b.up.railway.app/api/admin/users
```